

FACET2_S2E Update: major BMAD simulation improvements

Sergei Kladov

June 4, 2026

Overview

- **Upstream package:** FACET2_S2E.
- **Key changes:** Added a package "FACET2_S2E_Kladov", that corrects the default BMAD simulations, includes automatic lattice configuration according to an experimental setup, and introduces a number of QoL upgrades and control options.
- **Compatibility target:** Fully compatible with the current FACET2_S2E package.

The package has three functions that have the same name as in the original FACET2_S2E:

- `initializeTao()`,
- `trackBeam()`,
- `getBeamAtElement()`.

The original functions are not changed. To choose the function you need, use `FACET2_S2E.initializeTao()` or `FACET2_S2E_Kladov.initializeTao()`. `initializeTao()` and `trackBeam()` have wrappers; `getBeamAtElement()` does not. No other overlaps are present.

New wrapper for a start-to-end BMAD simulation

Function `get_tao_from_experiment()` acts as a new version of `FACET2_S2E.initializeTao()`. It initializes `tao`, loads the lattice and user settings, and sets up the collective effects and other general simulation parameters. Optionally, it loads the experimental lattice from a provided DAQ scan, loads a beam or creates a new one, edits its parameters including energy, edits the lattice energy based on how the beam is tracked, and runs the simulation. During the run, it can 1) change the bunch energy if `edited_bunch_energy_at_checkpoints_MeV` is supplied, or change the lattice energy multiple times (to track through the dipole properly if the desired lattice energy is not [125, 335, 4500, 10000] MeV) if `desired_POCs_MeV` is supplied.

Automatic lattice adjustments based on a DAQ scan

The user can load quadrupoles, sextupoles, correctors, and dipoles (actually, the dipoles are not loaded now because we never change them on the experiment) from a DAQ scan. This was tested against the liveSim (<https://confluence.slac.stanford.edu/spaces/FACET/pages/496468057/Bmad+live+modeling>) that loads quadrupoles and correctors. Both tools also load the cavity settings, but with remarks.

- **Important details of the DAQ load:**

- A user can choose to adjust energy (cavity settings) only (without touching the magnets).
- The default behavior for correctors is off. A user can adjust a coefficient by which all the correctors are multiplied.
- The phase set to all klystrons is the same (for a given cavity). But the EPICS databases suggest that in experiment there are two klystrons with $\pm$ large phase offset. It is disregarded here, in the same way as in the default FACET-S2E package.
- The klystron voltages are the same for a given cavity and are tuned to the experiment-default [125, 335, 4500, 10000] MeV. If other energies are used in the experiment, change them with the `desired_POCs_MeV` field of `get_tao_from_experiment()` or `run_initialized_sim()`.

Running an initialized simulation

- **Beam-based energy feedback**

This feature is available separately. Not in the `run_initialized_sim()` function.

- Added a framework that scales cavity voltages according to a tracked bunch and specified desired energy setpoints. The default setpoints are POCs (single-particle energies). The reference bunch can be passed; if not, a test bunch is created automatically. It is simulated four times to match four energy setpoints.

- **Beam energy edits on-the-go**

This feature is included in the `run_initialized_sim_edit_bunch_energy()` function.

- Changes the bunch energy at BX0FBEG, BC11CBEG, ENDL2F, ENDL3F_2.

- **Cavity adjustments between the bunching chicanes and the dogleg**

This feature is included in the `run_initialized_sim()` function. Is needed to simulate the dipoles properly if the lattice energy is different from the dipole-defaults, and if the user does not use "dg" or "db.field".

- Tracks the bunch in a lattice with the specified energies (`desired_POCs_MeV`) outside of the dogleg and chicanes, and with the dipole-default [125, 335, 4500, 10000] inside.

Basic (hidden) function organization improvements

- **Lattice initialization is separated from the beam introduction:**

- `UTILITY_quickstart.py`'s `initializeTao` now does not take a bunch as an argument. All related code parts are removed, and the function now has a clear structure focusing only on the basic edits of simulation methods and lattice.
- `UTILITY_quickstart.py`'s `trackBeam` now assumes that the `tao` parameter is provided with an already initialized beam. If not, it returns an error (from `tao`).
- Adding a bunch to `tao` is done in a separate function `setBeam()`. The function makes the necessary adjustments to the bunch: sets all z to 0, resamples the beam if needed, and centers the bunch by t if needed.

- **Control over collective effects:**

- `UTILITY_quickstart.py`'s `initializeTao` now takes both Longitudinal Space Charge (`lscTF`) and Coherent Synchrotron Radiation (`csrTF`) switches. The function enables or disables them separately based on the switches.
- The `Activate_CSR.tao` file was simplified. Now the CSR and SC switches (see point above) turn the calculations on or off everywhere. To change where they are turned on and where not, see the function from the next point.
- The `initializeTao` sets default methods and parameters of collective effect calculation, with the user being able to provide the grid sizes for 3d methods. This includes setting the SR and LR wakes to on. TODO: add an option to enable CSR only in the Nathan's default positions. TODO: add default behavior to `initializeTao` so that the grid size can take a single number for z slices in 1d methods.
- `get_tao_from_experiment` from the new file sets `sr_wakes_on` to **off** by default (favorable for my simulations, change to on if needed).

- **Single-particle energy:**

- Removed constants from the Nathan's framework ("setLinacsHelper()"). Now the cavity voltage is set automatically based on desired POCs between the cavities.
- Added automatic L0A and L0B energy adjustments based on a specified ratio. The ratio can be taken from an experiment database (see below).

Bunch handling improvements

- **General bunch edit function:**

- Tunes the bunch to have the specified bunch sizes and correlations or Twiss parameters.

- **Creates a new bunch if needed:**

- Can create a Gaussian bunch and edit it according to Twiss parameters / second moment matrices if needed. The parameters can be passed explicitly, or taken from a different provided bunch.

Analysis

- **Lattice element list**

Added a new function `get_element_array()`:

- Takes a pair of elements and prints the list of elements between them.
- Supports adjustable margins for shrinking or expanding the list.
- Displays elements matching the specified types, or, reversely, not matching the specified types.

- **Single-particle transport map**

Added new functions `get_rij()` and `get_tijk()` for the 1st and 2nd order maps between two elements.

- **Improved beam plots**

Added a new function `print_result()`:

- Displays a set of 2d plots, specified by a user. The default set is the longitudinal phase space (with t).
- Prints and returns the sizes of the bunch. The user can specify whether the full energy spread or slice energy spread is needed, as well as units (delta or eV).

- **1D scans**

Added a new function `make_1d_scan()`:

- Takes a lattice adjustment function and output generation function, makes a scan, and plots the result.
- Shipped with `make_comparison_dz_2nd_order()` function that can act as an output generation function. It compares the full simulation with $R + T$ (2nd order transport map).

Microbunching functions

These functions are shipped in a separate .py file (still in the FACET2_S2E_Kladov package).

- **Tracking**

Added a new function `make_modulated_bunch()`:

- Adds longitudinal modulations (to the density $\rho(z)$) to the supplied bunch with a given wavelength and amplitude.
- The distribution can be viewed with the `hist()` function (not to confuse with other hist functions, such as `matplotlib.pyplot.hist()`).

- **Analysis**

Added spectrum functions to calculate, view and analyze the Fourier of $\rho(z)$.

- The spectrum is presented in frequency units normalized to the length of the bunch.
- The spectrum analysis extracts the volume under the largest peak of the Fourier spectrum. The peak is chosen within the bounds specified by the user. These bounds should be provided in meters - the function transforms the wavelength to the normalized frequency automatically.

Small adjustments

Changed the `getBeamAtLocation` function: removed "drift_to.." functions and setting z to 0. Reason: bunch taken from simulation should reflect the simulation properly. If mean z , mean t , or z spread was introduced during a simulation, a user should now be able to debug it. These `drift_to` and setting z and $\langle t \rangle$ to 0 are performed at the **beginning** of a simulation run.

Included the .yaml lattice settings (that are supplied by the user on top of the defaults.yaml) in the `initializeTao()` function (and, therefore, in the `get_tao_from_experiment()` function). Reason:

if a user wants to change anything for a simulation, they will most likely need to change it for multiple simulations. Adding this to `initializeTao()` as an optional argument is a QoL upgrade.

Files Changed (Pull Request Checklist)

File / Directory	Summary of Change
<code>src/Experimental_functions/</code>	Include DATASET and other DAQ functions
<code>src/FACET2_S2E_Kladov/</code>	The new package
<code>examples/Kladov_BMAD_tutorial.ipynb</code>	A tutorial with example for the new package
<code>examples/Kladov_BMAD_dipole_showcase.ipynb</code>	A notebook showing how the dipoles are simulated
<code>setLattice_configs/</code>	Compressed oneBunch.yml (no factual changes)
<code>2024-10-22_oneBunch-Copy1.yml</code>	Same as Copy1, with a different L0B phase (an example for the tutorial)
<code>setLattice_configs/</code>	
<code>2024-10-22_oneBunch-Copy2.yml</code>	A temporary beam. This is the "scratch" directory from the default FACET-S2E
<code>temp_beam/temp.h5</code>	

How to Use (Quick Start)

1. **Install/build:** Follow the instructions of FACET2_S2E.
2. **Configure:** Import together with FACET2_S2E: `from Experimental_functions import *; import FACET2_S2E; import FACET2_S2E_Kladov;`
3. **Run:** `FACET2_S2E_Kladov.initializeTao()`.

Open Items / Future Work

- Add support for IMPACT + QPAD.

Description of individual functions

See the comments in the code